PROJECT REPORT

CIFAR-10 Image Classification using Convolutional Neural Networks (CNN)

Deep Learning · Computer Vision · Model Deployment

Submitted by
Sushant Tiwari

Tech Stack: Python · TensorFlow/Keras · NumPy · Scikit-learn · Matplotlib · Seaborn · Flask

1. Objective

The objective of this project is to design, train, and deploy a Convolutional Neural Network (CNN) capable of classifying 32×32 RGB images from the CIFAR-10 dataset into 10 distinct object categories — airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck — with a target test accuracy of 85% or higher. Beyond the modeling exercise, the project also demonstrates how a trained model can be productionized: the final CNN is wrapped in a Flask web application with a browser-based interface, so any image can be classified in real time rather than remaining confined to a notebook.

2. Dataset

CIFAR-10 is a widely used benchmark dataset in computer vision, consisting of 60,000 color images of size 32×32×3, split into 50,000 training images and 10,000 test images, evenly distributed across 10 mutually exclusive classes.

Attribute	Value	Notes
Source	cs.toronto.edu/~kriz/cifar.html	Loaded via tf.keras.datasets.cifar10
Training images	50,000	5,000 per class
Test images	10,000	1,000 per class
Image size	32 × 32 × 3	RGB, uint8 pixel values 0–255
Classes	10	Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck

3. Libraries Used

Library	Purpose
TensorFlow / Keras	Building, training, and saving the CNN
NumPy	Array and tensor manipulation
SciPy	Supports Keras image augmentation utilities
Scikit-learn	Classification report and confusion matrix
Matplotlib	Plotting sample images and training curves
Seaborn	Confusion matrix heatmap visualization
Flask	Serving the trained model as a web application
Pillow (PIL)	Reading and resizing uploaded images at inference time

4. Methodology

4.1 Data Understanding & Exploration

The CIFAR-10 dataset is loaded pre-split into training and test sets using Keras's built-in loader. Basic shape checks confirm 50,000 training images and 10,000 test images, each 32×32×3. A 5×5 grid of sample images is plotted alongside their true class labels to visually verify that the data and labels are correctly aligned before any modeling begins.

4.2 Data Preprocessing & Augmentation

- Normalization: pixel intensities are rescaled from the [0, 255] integer range to the [0, 1] float range by dividing by 255, which stabilizes and speeds up gradient-based training.
- Augmentation: an ImageDataGenerator applies random rotations (±15°), width/height shifts (10%), and horizontal flips on the fly during training. This exposes the model to a wider variety of viewpoints without needing additional labeled data, which reduces overfitting on the relatively small 32×32 images.

4.3 Model Development — CNN Architecture

The network follows a classic deep CNN design: three convolutional blocks of increasing filter depth (32 → 64 → 128), each pairing two 3×3 convolutions with batch normalization, followed by max pooling and dropout for regularization. This is followed by a dense classification head.

Block	Layers	Filters/Units	Regularization
Conv Block 1	2× Conv2D(3×3) + MaxPool	32	BatchNorm, Dropout 0.25
Conv Block 2	2× Conv2D(3×3) + MaxPool	64	BatchNorm, Dropout 0.25
Conv Block 3	2× Conv2D(3×3) + MaxPool	128	BatchNorm, Dropout 0.25
Dense Head	Flatten → Dense → Output	128 → 10	BatchNorm, Dropout 0.5

```
Input (32, 32, 3)
├─ Conv2D(32,3x3)→BatchNorm→Conv2D(32,3x3)→BatchNorm→MaxPool(2x2)→Dropout(0.25)
├─ Conv2D(64,3x3)→BatchNorm→Conv2D(64,3x3)→BatchNorm→MaxPool(2x2)→Dropout(0.25)
├─ Conv2D(128,3x3)→BatchNorm→Conv2D(128,3x3)→BatchNorm→MaxPool(2x2)→Dropout(0.25)
├─ Flatten
├─ Dense(128, relu)→BatchNorm→Dropout(0.5)
└─ Dense(10, softmax)
```

Optimizer: Adam. Loss function: Sparse Categorical Cross-Entropy. Learning-rate schedule: ReduceLROnPlateau (monitors validation loss, halves the learning rate after 3 epochs of no improvement, down to a floor of 1e-6).

4.4 Model Training & Evaluation

The model is trained for up to 30 epochs on augmented batches of 64 images, with the held-out test set used as validation data after every epoch. Training and validation accuracy/loss curves are plotted after training to visually check for overfitting — a validation curve that tracks closely with the training curve indicates the batch normalization and dropout strategy is working as intended.

Final evaluation is performed on the full 10,000-image test set, producing an overall test accuracy and test loss. This is complemented by two additional diagnostics:

- A classification report — per-class precision, recall, and F1-score — which highlights whether particular categories (e.g. cat vs. dog, automobile vs. truck) are harder for the model to distinguish.
- A confusion matrix heatmap, which visualizes exactly which classes are being confused with one another.

5. Results

The notebook (CIFAR-10.ipynb) and its script equivalent (train_model.py) produce the following artifacts on each run: final training/validation accuracy and loss, an accuracy/loss curve plot, a full classification report, and a confusion matrix heatmap.

Note: Because CIFAR-10 (~170 MB) must be downloaded at training time, the exact accuracy, loss values, and plots below should be taken from your own executed run of CIFAR-10.ipynb / train_model.py (they are printed and plotted automatically at the end of training). Paste your run's final numbers and screenshots into this section before submitting, so the report reflects your actual experiment output rather than a generic estimate.

Metric	Value (fill in after training)
Final Training Accuracy	_____ %
Final Validation Accuracy	_____ %
Test Accuracy	_____ % (target: 85%)
Test Loss	_____
Overfitting Check	Train/Val accuracy gap: _____ %

Typical behavior for this architecture on CIFAR-10: with batch normalization, progressive dropout, and rotation/shift/flip augmentation, training and validation accuracy tend to track closely (indicating limited overfitting), and the most common confusions in the classification report/confusion matrix are between visually similar classes such as cat ↔ dog and automobile ↔ truck, which is consistent with published CIFAR-10 benchmarks for CNNs of this depth.

6. Deployment — Web Application

To move beyond a static notebook, the trained model is saved (model/cifar10_cnn_model.h5) and served through a Flask backend (main.py). A browser-based interface (templates/index.html) lets a user drag and

drop any image; the app resizes it to 32×32, normalizes it the same way as during training, and returns the predicted class along with a confidence breakdown across all 10 categories.

- main.py — Flask routes for the home page, prediction endpoint, and health check.
- templates/index.html — upload UI with live confidence bars per class.
- test_app.py — automated test suite (6 tests) covering page load, health check, valid/invalid predictions, and probability sanity checks.
- test_samples/generate_test_samples.py — utility to pull real CIFAR-10 test images for quick manual testing of the app.
- launch.sh / launch.bat — one-command setup and launch scripts for Linux/Mac and Windows.

7. Conclusion

This project implements a complete, end-to-end CNN-based image classification pipeline for CIFAR-10 — from data loading and augmentation, through a regularized convolutional architecture with batch normalization and progressive dropout, to thorough evaluation via accuracy curves, a classification report, and a confusion matrix. Beyond the model itself, the project demonstrates practical ML engineering by productionizing the trained network behind a Flask API and web interface, complete with automated tests and cross-platform launch scripts, turning a notebook experiment into a usable application.

Potential extensions include deeper or residual (ResNet-style) architectures, transfer learning from ImageNet-pretrained backbones, and further augmentation or ensembling strategies to push test accuracy beyond the 85% target with greater consistency.

— Prepared by Sushant Tiwari